



MULTI-CLIENTES

CHADWICK CARRETO ARELLANO

18 DE SEPTIEMBRE DE 2025

ROSALES BARRAZA ERICK EFREN
7CM1

Múltiples clientes en un solo servidor e interfaz simplificada

Tercera parte: identificación de clientes, bitácora compartida y rediseño de la interfaz

Antecedentes

Las dos partes anteriores del proyecto ya dejaban correr la simulación de pedidos, planificador Round Robin e inventario compartido detrás de dos formas de acceso: un servidor web con WebSocket y un servidor de sockets TCP con un cliente de consola. En ambos casos, el servidor guardaba una colección de conexiones activas y les mandaba el mismo estado a todas, así que en los hechos ya soportaban varios clientes al mismo tiempo.

Lo que faltaba era que esa concurrencia se viera. Nada en la bitácora ni en la interfaz distinguía qué cliente había hecho qué acción, así que dos personas conectadas a la vez no tenían forma de notar la presencia de la otra más allá de ver que el estado cambiaba solo. Además, la interfaz web había ido creciendo con cada parte del proyecto, y para este punto mostraba bastantes paneles a la vez sin resaltar cuál era, en cualquier momento dado, el proceso que de verdad tenía la CPU. Por último, el texto orientado al usuario usaba voseo argentino, cuando el público de este proyecto habla español de México.

Planteamiento del problema

El pedido tenía tres partes. Primero, identificar a cada cliente conectado, sin importar si llega por WebSocket o por socket TCP, y usar esa identificación para etiquetar sus acciones en la misma bitácora que ya usaba el planificador. Segundo, simplificar la interfaz web y hacer más visible el proceso que está corriendo en cada turno, sin perder la información que ya mostraban el buffer, el inventario y los semáforos. Tercero, corregir el registro del español en todos los textos dirigidos al usuario.

Solución

Cada servidor, tanto el de WebSocket como el de sockets TCP, ahora asigna un número de cliente consecutivo a cada conexión entrante y lo guarda junto al socket. Ese número se manda de vuelta al cliente en un mensaje de bienvenida, y se usa para etiquetar en la bitácora cada acción que ese cliente dispare: agregar un proceso, enviar un pedido, pausar, o cambiar el quantum o la velocidad. La simulación también lleva ahora un contador de clientes conectados, que sube y baja con cada conexión y desconexión, y viaja en cada snapshot de estado.

Para que esas acciones lleguen a la misma bitácora que ya llenaba el planificador con sus propios eventos, se agregó un método público en el planificador para escribir una línea de texto arbitraria, y un método equivalente en la simulación que lo expone hacia afuera. Así, un evento disparado por un cliente externo y un evento generado internamente por un cambio de contexto terminan en la misma lista y se ven mezclados en orden, tal como ocurrieron.

Del lado de la interfaz, se agregó una tarjeta que muestra, en grande, qué proceso tuvo la CPU en el último turno, cuántas instrucciones usó de su quantum, y si terminó, se bloqueó o simplemente agotó su turno. El botón de pausar y el de reanudar se unieron en uno solo que cambia de texto según el estado. El primer proceso de la cola de listos, el que va a recibir la CPU a continuación, queda resaltado con un borde. Y la bitácora ahora colorea cada línea según lo que describe: bloqueos en rojo, despachos e desbloqueos en verde, cambios de contexto en azul.

Implementación

- scheduler.js gana un método log() que expone hacia afuera el mismo mecanismo interno que ya usaba para registrar sus propios eventos.
- simulation.js agrega logEvent(), junto con connectedClients y los métodos clientConnected() y clientDisconnected() para llevar la cuenta de conexiones activas; connectedClients ahora viaja en cada snapshot.
- server.js y socket-server.js asignan un clientId consecutivo a cada conexión nueva, lo mandan en un mensaje de bienvenida, y llaman a sim.logEvent() con ese identificador en cada comando que procesan, así como al conectarse y desconectarse.
- socket-client.js corrigió, de paso, un error real: si se cerraba la entrada estándar del cliente, el proceso se quedaba corriendo sin cerrar el socket ni terminar; ahora el cierre de la línea de comandos también cierra la conexión.
- public/index.html y public/client.js se reorganizaron: la tarjeta de CPU, un solo botón de pausa y reanudación, la cola de listos con el siguiente proceso resaltado, y una bitácora con colores por tipo de evento.
- Los textos dirigidos al usuario, tanto en la interfaz web como en el cliente de consola, se corrigieron de voseo argentino a español de México.

El código de identificación de clientes y las llamadas a logEvent() quedaron organizados así:

simulation.js — logEvent y contador de clientes conectados

```
logEvent(msg) {
    this.scheduler.log(msg);
  }

  clientConnected() {
    this.connectedClients += 1;
    return this.connectedClients;
  }

  clientDisconnected() {
    this.connectedClients = Math.max(0, this.connectedClients - 1);
  }
```

server.js / socket-server.js — cada comando se etiqueta con el cliente que lo mandó

```
case 'addProducer': {
  const id = sim.addProducer();
  sim.logEvent(`Cliente ${clientId} agrega cliente automatico ${id}`);
  break;
}
case 'submitOrder':
  try {
    const id = sim.addCustomOrder(msg.product, msg.qty);
    sim.logEvent(`Cliente ${clientId} envia pedido manual ${id} (${msg.qty}x
${msg.product})`);
  } catch (err) {
    send(socket, { type: 'error', message: err.message });
```

```
    return;
  }
  break;
```

Resultados

- Con dos pestañas del navegador abiertas, cada una mostró un número de cliente distinto y el mismo conteo de clientes conectados en tiempo real.
- Una acción disparada desde la segunda pestaña, agregar un cliente automático, apareció en la bitácora de la primera como "Cliente 2 agrega cliente automatico P5", visible para ambas sin refrescar nada.
- Con dos clientes de consola conectados al servidor de sockets TCP al mismo tiempo, cada uno recibió su propio número de cliente y ambos vieron el conteo de conectados en dos, además del mismo buffer y el mismo inventario compartido.
- La nueva tarjeta de CPU mostró en pantalla, por ejemplo, "C3, agotó su quantum, 3/3 instrucciones", con una barra de progreso, y el proceso siguiente en la cola de listos quedó marcado con un borde azul.
- El botón único de pausa y reanudación cambió de texto correctamente y la bitácora registró la acción como "Cliente 1 pausa la simulación".
- El texto de estado ahora dice "Eres el cliente 1" en vez de la forma con voseo usada antes.

Conclusiones

Haber separado desde el inicio la lógica de la simulación de su capa de transporte volvió a rendir frutos: identificar clientes y etiquetar sus acciones solo necesitó un método nuevo en el planificador y un poco de código en cada servidor, sin tocar el algoritmo de planificación, los semáforos ni el inventario.

Que el sistema funcionara correctamente con varios clientes no bastaba por sí solo: hacía falta que esa concurrencia se pudiera ver, tanto en la bitácora compartida como en una interfaz que señalara con claridad qué proceso tiene la CPU en cada momento. Simplificar la interfaz y agregar la tarjeta de CPU resultó tan importante para entender el planificador como el propio algoritmo de Round Robin.

Corregir el registro del idioma es un detalle pequeño frente a la lógica de sincronización, pero importa igual: un sistema pensado para un público de habla hispana en México debía sonar como se habla ahí, no como en otro país de la región.